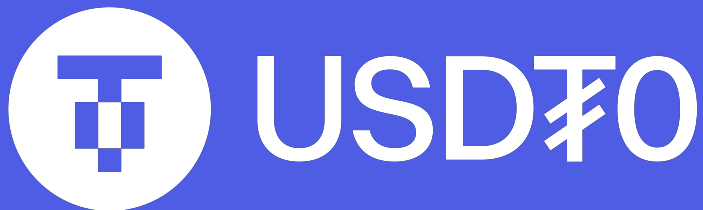


XAUT0 TON OFT Audit



August 14, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Cross-Chain Flow	5
Key Differences from a Standard Jetton	6
Security Model and Trust Assumptions	6
Privileged Roles	7
Medium Severity	8
M-01 Token Admin Cannot Claim TON	8
Low Severity	8
L-01 Silent Burn via Zero forward_ton_amount in jetton-minter::receive_jettons	8
Notes & Additional Information	9
N-01 Incorrect Comment in Opcode Execution	9
N-02 Misleading Error Code for Master-Only Check in jetton-wallet::burn_jettons	9
N-03 Unrestricted EVENT Opcode in OFT Contract Allows Arbitrary Event Emission	10
N-04 Duplicate Event Risk in Setter Functions	10
N-05 Non-Bounceable Transfer from Jetton-Minter to OFT Can Burn User Funds	11
Conclusion	12

Summary

Type	Stablecoins	Total Issues	7 (2 resolved)
Timeline	From 2025-06-18 To 2025-07-11	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	1 (1 resolved)
		Low Severity Issues	1 (0 resolved)
		Notes & Additional Information	5 (1 resolved)

Scope

OpenZeppelin audited the [Everdawn-Labs/xaut-ton-everdawn](https://github.com/Everdawn-Labs/xaut-ton-everdawn) repository at commit [1d36237](#).

In scope were the following files:

```
src
├── BamOFT
│   ├── handler.fc
│   └── main.fc
├── Token
│   ├── gas.fc
│   ├── jetton-minter.fc
│   ├── jetton-utils.fc
│   ├── jetton-wallet.fc
│   ├── jetton.tlb
│   ├── op-codes.fc
│   ├── stdlib.fc
│   └── workchain.fc
├── TokenAdmin
│   ├── handler.fc
│   ├── interface.fc
│   ├── main.fc
│   └── storage.fc
├── modules
│   ├── OFT
│   │   ├── consts.fc
│   │   ├── handlerOFT.fc
│   │   ├── interface.fc
│   │   └── storage.fc
│   └── oApp
│       ├── handlerOApp.fc
│       ├── interface.fc
│       └── storage.fc
└── oftStorages
    ├── classAutoload.fc
    └── main.fc
```

System Overview

XAUt0 is a bridged version of Tether's XAUt, designed to extend XAUt availability to blockchains where it is not natively issued. Powered by LayerZero's Omnichain Fungible Token (OFT) standard, it maintains a 1:1 relationship between locked XAUt on Ethereum and the minted tokens on supported chains. On TON, XAUt0 is implemented as a Jetton using the Jetton fungible-token standard for local accounting and integrates LayerZero's Omnichain Fungible Token (OFT) framework. This allows the Jetton supply on TON to expand or contract as users bridge XAUt0 to and from other LayerZero-enabled networks.

The Ton-specific design splits the logic across **three main smart contracts**:

Contract	Description
Jetton Wallet	Modified <code>jetton-wallet</code> that supports OFT transfers
Jetton Minter	Modified version of a <code>jetton-minter</code> contract that supports OFT transfers
BamOFT	Bridges to LayerZero: it receives <code>transfer_notification</code> events from the jetton-minter, packages them into LayerZero messages, and mints or releases XAUt0 when the <code>lzReceive</code> callbacks arrive.

Cross-Chain Flow

- Send (TON → other chains):

1. The user's **Jetton Wallet** sends Jettons to the **Jetton Minter**.
2. **Jetton Minter** burns the Jettons and sends a `transfer_notification` to **BamOFT**.
3. **BamOFT** picks up the notification and calls LayerZero `lzSend`.

- Receive (other chains → TON):

1. LayerZero delivers an `lzReceive` message to **BamOFT**, which on TON uses a two-phase commit (prepare then execute) instead of the single-phase flow used on EVM chains.
2. **BamOFT** sends a `mint` message to the **Jetton Minter**.

3. **Jetton Minter** mints the specified Jetton amount to the recipient's **Jetton Wallet**.

Key Differences from a Standard Jetton

The following key differences between the standard Jetton Minter and the Modified Jetton Minter were identified:

- **Extra field in Jetton Minter:** A `mint_to_authority` address (BamOFT) is stored alongside `admin_address`.
- **Bridge burn path:** Minter implements a `receive_jettons` function. It receives incoming Jettons from wallets and sends a `transfer_notification` to BamOFT.
- **Mint permissions:** The `mint` opcode now accepts calls from BamOFT as well as the admin.
- **Wallet burn control:** `burn_jettons` in the wallet can only be invoked by the master (minter), not by the wallet owner.
- **Master initiated transfers:** The master can send Jettons out of any wallet, bypassing normal send/receive locks.
- **Send/receive lock bits in the wallet:** A new `status` field lets the minter lock outgoing (`bit 0`) or incoming (`bit 1`) transfers. The master address can still move funds even when a lock has been set.

Security Model and Trust Assumptions

During the audit, the following observations and trust assumptions were made:

- **Forwarding success:** Every `transfer_notification` sent from the Jetton Minter to BamOFT is expected to clear the TVM *compute* phase. If BamOFT reverts during that phase because of an invalid payload, a contract pause, a rate-limit check, or an out-of-gas condition, the message is discarded after `total_supply` has already been reduced, so the user's tokens are permanently burned.

- **Client side preparation and fee coverage:** Messages that are malformed or under-funded revert, and the protocol does not add missing fees or refund them. Off-chain tooling (front end or SDK) must:
 - construct TL-B payloads correctly
 - include enough TON to cover the entire OFT send flow. Gas must pay for the wallet-to-minter transfer, the minter-to-BamOFT forward, and any later steps in the process
- **Admin authority:** The admin of the Jetton Minter and the admin of the OFT contract can upgrade code and data without on-chain guards. A compromised or malicious admin could deploy arbitrary logic or change critical parameters. The security model assumes these administrators act honestly and will perform multiple sanity checks before upgrading code or data.
- **LayerZero dependency:** Correct operation across chains depends on LayerZero endpoint contracts and libraries behaving as specified. The model assumes these external components work as intended.

Privileged Roles

Throughout the codebase, the following privileged roles were identified:

- **Jetton Minter Admin:** It can mint to any wallet, burn or transfer tokens via `call_to` from any user wallet, lock or unlock wallets, change metadata URI, change or drop the admin, and upgrade minter code and data.
- **OFT Owner:** It can pause or unpause the bridge, deploy channels and connections, force-abort transfers, set peers and enforced options, update LayerZero config, transfer or claim ownership, and upgrade code.
- **OFT Admin:** It can pause and unpause the OFT contract.

Medium Severity

M-01 Token Admin Cannot Claim TON

The `claimTon` admin function pushes a `payment action` to the list of actions that has type `action::payment::name`. However, the `_executeAction` function of the `TokenAdmin` handler does not include logic to process this action type and will instead throw an `invalidActionType` error. This makes it impossible for the admin to retrieve any excess TON stored in the contract.

Consider updating the `_executeAction` function to account for the payment action type.

Update: Resolved in [pull request #2](#) at [commit f897c88](#). The Everdawn team stated:

*Fixed. We have updated `_executeAction` to handle `action::payment::name`. Note that this is an admin-only contract that is currently **not deployed** as the jetton's admin. So, no action is required.*

Low Severity

L-01 Silent Burn via Zero `forward_ton_amount` in `jetton-minter::receive_jettons`

The `receive_jettons` function silently burns tokens whenever the incoming transfer sets `forward_ton_amount` to 0. The minter wallet credits itself with the received `jetton_amount` and updates `total_supply`. However, because no `transfer_notification` is emitted, the OFT contract never receives the tokens, and the function simply saves state and returns.

Therefore, a transfer becomes equivalent to a burn if the sender omits the forwarding TON, even though explicit burning is restricted to the Jetton master and should not be available to users. This silent-burn path not only leaves users at risk of permanently losing their Jettons but also clouds the protocol's supply guarantees, since tokens can be burned without the master's

explicit burn routine. If users are not meant to burn tokens, the wallet should bounce any transfer with a zero `forward_ton_amount`.

Consider bouncing any internal transfer whose `forward_ton_amount` is 0 so that the transaction fails visibly instead of silently burning tokens. Alternatively, consider clearly documenting this behavior. Doing so will help integrators understand that a `forward_ton_amount` of 0 turns a transfer into a user-initiated burn.

Update: Acknowledged, not resolved. The Everdawn team stated:

| We have decided not to fix this as it only affects the UX.

Notes & Additional Information

N-01 Incorrect Comment in Opcode Execution

When a `transfer_notification` is sent to the BamOFT handler, the message type must be `OFTSend::NAME` for the `sendOFT` call to be performed. If not, [no actions are performed](#), which results in any remaining message value being [refunded to the origin](#). However, this behavior is not correctly documented, with the comment stating that the empty actions return [will not ever be hit](#).

Consider clarifying the comment to properly describe the intended process when a `transfer_notification` is sent with an incorrect message type.

Update: Acknowledged, not resolved.

N-02 Misleading Error Code for Master-Only Check in `jetton-wallet::burn_jettons`

The `burn_jettons` function [validates](#) that the caller is the Jetton minter by throwing `error::not_owner` when the check fails. This conflates two separate concepts: wallet ownership and master-minter authority. Since the codebase already [defines](#) `error::not_minter` for “only master may call” situations, reusing `error::not_owner` obscures the real cause of the reversion and may cause confusion for integrators.

Consider replacing `error::not_owner` with the semantically correct `error::not_minter`.

Update: Acknowledged, not resolved. The Everdawn team stated:

This implementation matches the stablecoin-contract standard. We do not want to change the behavior.

N-03 Unrestricted EVENT Opcode in OFT Contract Allows Arbitrary Event Emission

The `checkPermissions()` function [returns](#) without any validation when `op` equals `BaseInterface::OP::EVENT`. As a result, the subsequent logic in `main()` always reaches the logic that [emits](#) the event. Therefore, any external account can send a message with `op = EVENT` and force the contract to emit an event, even if the contract is uninitialized or the caller has no special privileges. Allowing anyone to emit events lets attackers spam the event log, potentially triggering off-chain systems that rely on authentic contract events and undermining trust in emitted data.

Consider adding an authorization check (e.g., verifying the caller is the OFT contract or another whitelisted entity) before processing `OP::EVENT`. Alternatively, consider clearly documenting the behavior and explaining how integrators can distinguish legitimate events (e.g., by validating the sender address).

Update: Resolved. The Everdawn team stated:

This is by design. Events are indexed by src and dst, so it is only an off-chain concern if clients index by dst alone.

N-04 Duplicate Event Risk in Setter Functions

When a setter function does not check if the value being set is different from the existing one, it becomes possible to set the same value repeatedly, creating a possibility for event spamming. Repeated emission of identical events can also confuse off-chain clients.

Throughout the codebase, multiple instances of code allowing for duplicate event emissions were identified:

- `BamOFT::Pause`
- `BamOFT::UnPause`

- `BamOFT::setRateLimit`

Consider adding a check to revert the transaction if the value being set is identical to the existing value.

Update: Acknowledged, not resolved. The Everdawn team stated:

All affected functions are admin-only. This means that the admin can event-spam its own listeners.

N-05 Non-Bounceable Transfer from Jetton-Minter to OFT Can Burn User Funds

The `receive_jettons` function in the Jetton-Minter subtracts the received Jettons from `total_supply` and forwards them to OFT contract in a `non-bounceable transfer_notification` message. The selected send-mode only bounces when OFT fails during its action phase.

Any compute-phase error, such as an invalid payload, rate-limit rejection, contract pause, or out-of-gas condition, reverts the transfer without a bounce and effectively burns the tokens. Since the supply has already been reduced, the user's tokens are lost forever and the on-chain supply of TON no longer matches the USDT locked on Ethereum. The risk is amplified by TON's asynchronous execution: a pause or rate-limit update sent after the user's transfer can still be processed first if it lands in an earlier block, triggering the failure scenario.

Consider sending the `transfer_notification` as a bounceable message. This way, if OFT were to fail in its compute phase, the value is returned automatically, letting the Minter restore `total_supply` and refund the Jettons to the user.

Update: Acknowledged, not resolved. The compute phase extends beyond the `try/catch` block. As such, out-of-gas or invalid-payload errors can occur before the `try/catch` block executes. The Everdawn team stated:

`sendOFT` is wrapped in a `try/catch` block. Failures trigger refund and user funds are not lost.

Conclusion

XAUt0 implements a LayerZero OFT-enabled Jetton on TON that mirrors Ethereum XAUt via cross-chain minting and burning. The audit uncovered one medium-severity issue where excess TON can become irretrievable in the admin contract. In addition, several low-severity findings and informational notes were also reported. While the codebase was found to be well-structured, it could benefit from more descriptive code comments. The Everdawn team is appreciated for being highly responsive and providing valuable support throughout the audit process.